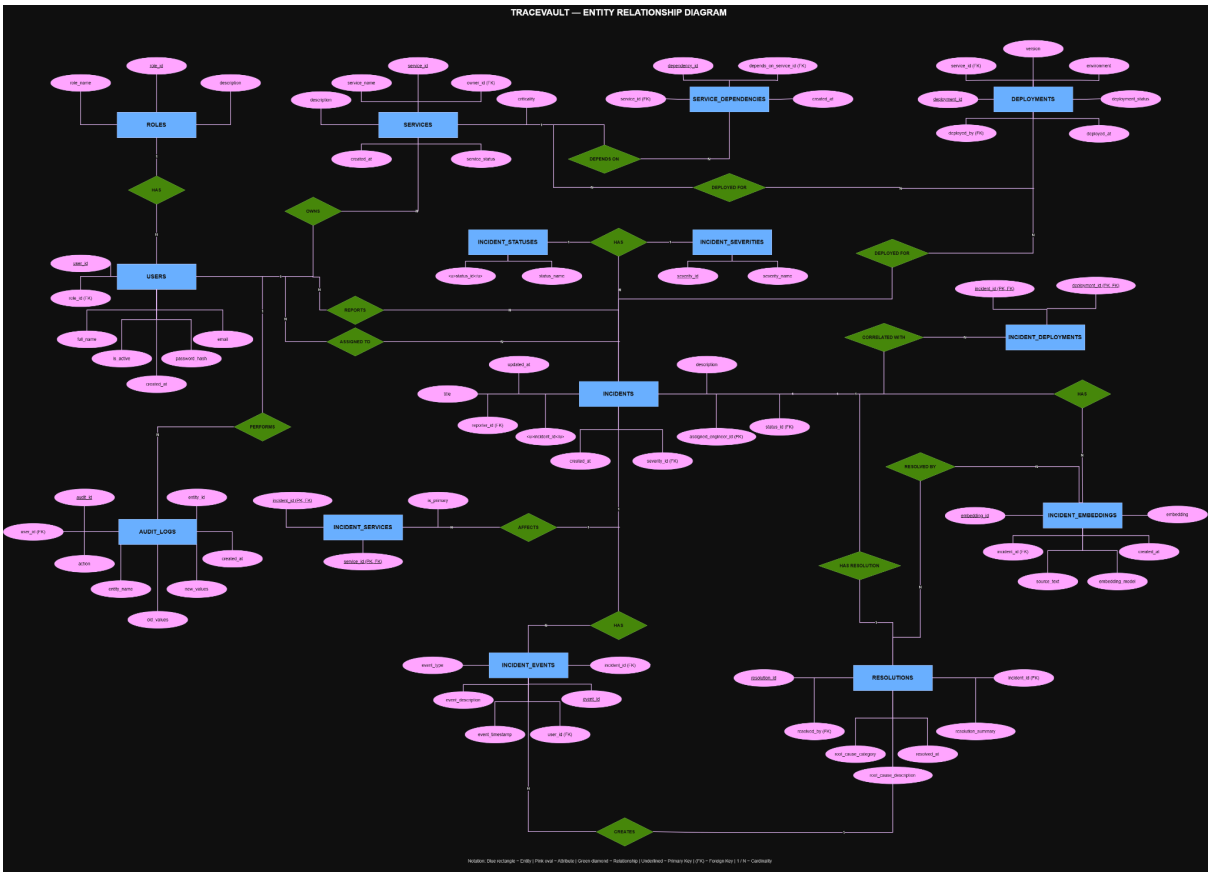
TraceVault: Incident Causality and Resolution Intelligence Platform

Name: Jaanhavi Ashwin Kher, Kushi Niranjana

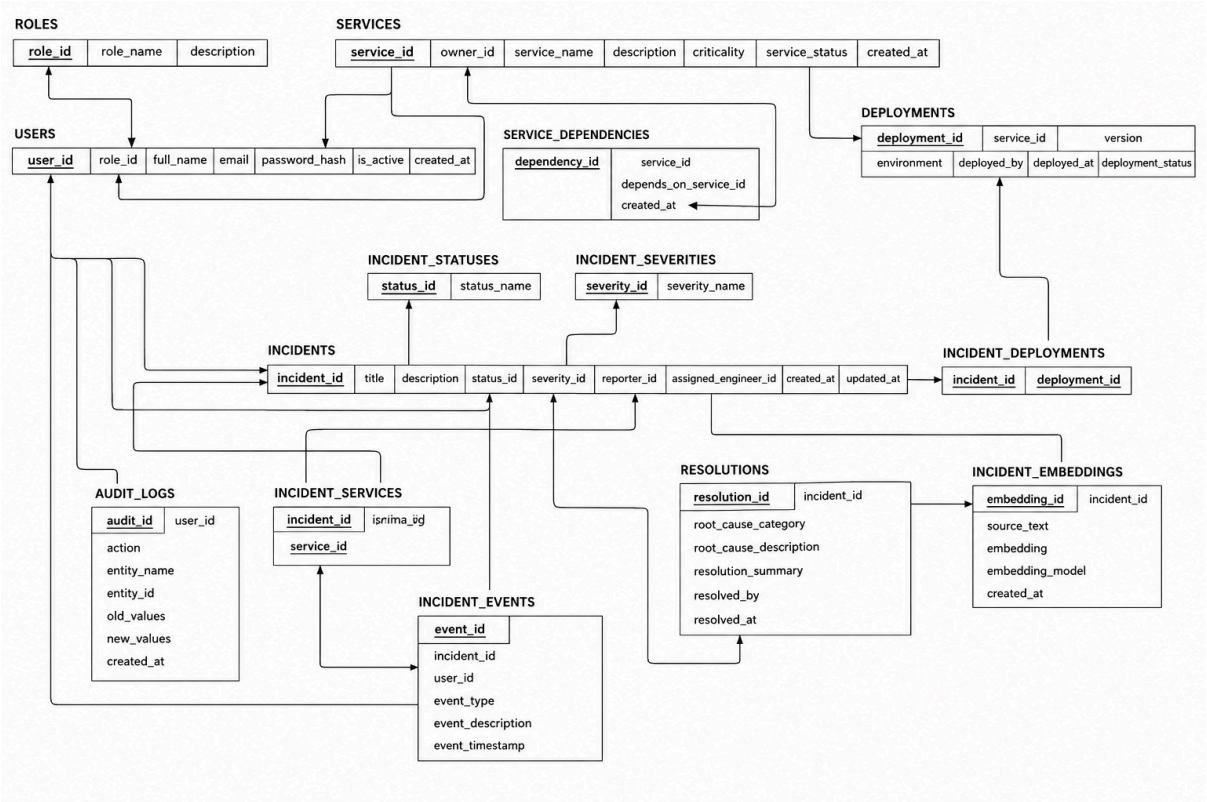
SRN:PES2UG24CS199, PES2UG24CS245

SECTION: D

ER DIAGRAM


TRACEVAULT — ENTITY RELATIONSHIP DIAGRAM
ROLES
SERVICES
SERVICE_DEPENDENCIES
DEPLOYMENTS
USERS
INCIDENT_STATUSES
INCIDENT_SEVERITIES
INCIDENTS
INCIDENT_DEPLOYMENTS
AUDIT_LOGS
INCIDENT_SERVICES
INCIDENT_EMBEDDINGS
INCIDENT_EVENTS
RESOLUTIONS
HAS
OWNS
DEPENDS ON
DEPLOYED FOR
REPORTS
ASSIGNED TO
CORRELATED WITH
PERFORMS
AFFECTS
HAS RESOLUTION
RESOLVED BY
CREATES

RELATIONAL SCHEMA



DDL COMMANDS

```
CREATE TABLE roles (
    role_id SERIAL PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255)
);
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    role_id INT NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_users_role
        FOREIGN KEY (role_id)
        REFERENCES roles(role_id)
);
CREATE TABLE services (
    service_id SERIAL PRIMARY KEY,
    service_name VARCHAR(150) NOT NULL UNIQUE,
```

```
    description TEXT,

    criticality VARCHAR(20) NOT NULL,
    service_status VARCHAR(20) NOT NULL,

    owner_id INT,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_service_criticality
        CHECK (criticality IN
            ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),

    CONSTRAINT chk_service_status
        CHECK (service_status IN
            ('ACTIVE', 'INACTIVE', 'MAINTENANCE')),

    CONSTRAINT fk_service_owner
        FOREIGN KEY (owner_id)
        REFERENCES users(user_id)
        ON DELETE SET NULL
);
CREATE TABLE service_dependencies (
    dependency_id SERIAL PRIMARY KEY,

    service_id INT NOT NULL,
    depends_on_service_id INT NOT NULL,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_dependency_service
        FOREIGN KEY (service_id)
        REFERENCES services(service_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_dependency_depends_on
        FOREIGN KEY (depends_on_service_id)
        REFERENCES services(service_id)
        ON DELETE CASCADE,

    CONSTRAINT chk_no_self_dependency
        CHECK (service_id <> depends_on_service_id),

    CONSTRAINT uq_service_dependency
        UNIQUE (service_id, depends_on_service_id)
);

CREATE TABLE incident_statuses (
```

```
    status_id SERIAL PRIMARY KEY,
    status_name VARCHAR(30) NOT NULL UNIQUE
);
CREATE TABLE incident_severities (
    severity_id SERIAL PRIMARY KEY,
    severity_name VARCHAR(20) NOT NULL UNIQUE
);
CREATE TABLE incidents (
    incident_id SERIAL PRIMARY KEY,

    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,

    status_id INT NOT NULL,
    severity_id INT NOT NULL,

    reporter_id INT NOT NULL,
    assigned_engineer_id INT,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_incident_status
        FOREIGN KEY (status_id)
        REFERENCES incident_statuses(status_id),

    CONSTRAINT fk_incident_severity
        FOREIGN KEY (severity_id)
        REFERENCES incident_severities(severity_id),

    CONSTRAINT fk_incident_reporter
        FOREIGN KEY (reporter_id)
        REFERENCES users(user_id),

    CONSTRAINT fk_incident_engineer
        FOREIGN KEY (assigned_engineer_id)
        REFERENCES users(user_id)
        ON DELETE SET NULL
);
CREATE TABLE incident_services (
    incident_id INT NOT NULL,
    service_id INT NOT NULL,

    is_primary BOOLEAN NOT NULL DEFAULT FALSE,

    PRIMARY KEY (incident_id, service_id),

    CONSTRAINT fk_incident_services_incident
```

```
    FOREIGN KEY (incident_id)
    REFERENCES incidents(incident_id)
    ON DELETE CASCADE,

  CONSTRAINT fk_incident_services_service
    FOREIGN KEY (service_id)
    REFERENCES services(service_id)
    ON DELETE CASCADE
);
CREATE TABLE incident_events (
  event_id SERIAL PRIMARY KEY,

  incident_id INT NOT NULL,
  user_id INT,

  event_type VARCHAR(100) NOT NULL,
  event_description TEXT NOT NULL,

  event_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT fk_event_incident
    FOREIGN KEY (incident_id)
    REFERENCES incidents(incident_id)
    ON DELETE CASCADE,

  CONSTRAINT fk_event_user
    FOREIGN KEY (user_id)
    REFERENCES users(user_id)
    ON DELETE SET NULL
);
CREATE TABLE deployments (
  deployment_id SERIAL PRIMARY KEY,

  service_id INT NOT NULL,

  version VARCHAR(100) NOT NULL,
  environment VARCHAR(50) NOT NULL,

  deployed_by INT,

  deployed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

  deployment_status VARCHAR(30) NOT NULL,

  CONSTRAINT fk_deployment_service
    FOREIGN KEY (service_id)
    REFERENCES services(service_id)
    ON DELETE CASCADE,
```

```
    CONSTRAINT fk_deployment_user
        FOREIGN KEY (deployed_by)
        REFERENCES users(user_id)
        ON DELETE SET NULL,

    CONSTRAINT chk_deployment_status
        CHECK (deployment_status IN
            ('SUCCESS', 'FAILED', 'ROLLED_BACK'))
);
CREATE TABLE incident_deployments (
    incident_id INT NOT NULL,
    deployment_id INT NOT NULL,

    PRIMARY KEY (incident_id, deployment_id),

    CONSTRAINT fk_incident_deployment_incident
        FOREIGN KEY (incident_id)
        REFERENCES incidents(incident_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_incident_deployment_deployment
        FOREIGN KEY (deployment_id)
        REFERENCES deployments(deployment_id)
        ON DELETE CASCADE
);
CREATE TABLE resolutions (
    resolution_id SERIAL PRIMARY KEY,

    incident_id INT NOT NULL UNIQUE,

    root_cause_category VARCHAR(100) NOT NULL,
    root_cause_description TEXT NOT NULL,

    resolution_summary TEXT NOT NULL,

    resolved_by INT,

    resolved_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_resolution_incident
        FOREIGN KEY (incident_id)
        REFERENCES incidents(incident_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_resolution_user
        FOREIGN KEY (resolved_by)
        REFERENCES users(user_id)
```

```
    ON DELETE SET NULL
);
CREATE TABLE incident_embeddings (
    embedding_id SERIAL PRIMARY KEY,

    incident_id INT NOT NULL,

    source_text TEXT NOT NULL,

    embedding VECTOR(1536) NOT NULL,

    embedding_model VARCHAR(100) NOT NULL,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_embedding_incident
        FOREIGN KEY (incident_id)
        REFERENCES incidents(incident_id)
        ON DELETE CASCADE
);
CREATE TABLE audit_logs (
    audit_id SERIAL PRIMARY KEY,

    user_id INT,

    action VARCHAR(100) NOT NULL,

    entity_name VARCHAR(100) NOT NULL,
    entity_id INT NOT NULL,

    old_values JSONB,
    new_values JSONB,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_audit_user
        FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON DELETE SET NULL
);
CREATE INDEX idx_incidents_status
ON incidents(status_id);

CREATE INDEX idx_incidents_severity
ON incidents(severity_id);

CREATE INDEX idx_incidents_reporter
ON incidents(reporter_id);
```

```
CREATE INDEX idx_incidents_assigned_engineer
ON incidents(assigned_engineer_id);

CREATE INDEX idx_incidents_created_at
ON incidents(created_at);

CREATE INDEX idx_incident_events_incident_time
ON incident_events(incident_id, event_timestamp);

CREATE INDEX idx_deployments_service_date
ON deployments(service_id, deployed_at);

CREATE INDEX idx_service_dependencies_service
ON service_dependencies(service_id);

ALTER TABLE incidents
ADD COLUMN priority_score INT DEFAULT 0;

ALTER TABLE incidents
ADD CONSTRAINT chk_priority_score
CHECK (priority_score BETWEEN 0 AND 100);

CREATE TABLE temporary_test_table (
   test_id SERIAL PRIMARY KEY,
   test_name VARCHAR(100)
);

DROP TABLE temporary_test_table;
```

DDL Commands Execution Screenshots

Query | Query History

```sql
CREATE TABLE roles (
role_id SERIAL PRIMARY KEY,
role_name VARCHAR(50) NOT NULL UNIQUE,
description VARCHAR(255)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 205 msec.
```

Query | Query History

```sql
CREATE TABLE users (
user_id SERIAL PRIMARY KEY,
role_id INT NOT NULL,
full_name VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE,
password_hash VARCHAR(255) NOT NULL,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_users_role
FOREIGN KEY (role_id)
REFERENCES roles(role_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 81 msec.
```

Query | Query History

```sql
CREATE TABLE services (
service_id SERIAL PRIMARY KEY,
service_name VARCHAR(150) NOT NULL UNIQUE,
description TEXT,
criticality VARCHAR(20) NOT NULL,
service_status VARCHAR(20) NOT NULL,
owner_id INT,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_service_criticality
CHECK (criticality IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
CONSTRAINT chk_service_status
CHECK (service_status IN ('ACTIVE', 'INACTIVE', 'MAINTENANCE')),
CONSTRAINT fk_service_owner
FOREIGN KEY (owner_id)
REFERENCES users(user_id)
ON DELETE SET NULL
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 87 msec.
```

Query | Query History

```sql
CREATE TABLE service_dependencies (
dependency_id SERIAL PRIMARY KEY,
service_id INT NOT NULL,
depends_on_service_id INT NOT NULL,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_dependency_service
FOREIGN KEY (service_id)
REFERENCES services(service_id)
ON DELETE CASCADE,
CONSTRAINT fk_dependency_depends_on
FOREIGN KEY (depends_on_service_id)
REFERENCES services(service_id)
ON DELETE CASCADE,
CONSTRAINT chk_no_self_dependency
CHECK (service_id <> depends_on_service_id),
CONSTRAINT uq_service_dependency
UNIQUE (service_id, depends_on_service_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE incident_statuses (
status_id SERIAL PRIMARY KEY,
status_name VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE incident_severities (
severity_id SERIAL PRIMARY KEY,
severity_name VARCHAR(20) NOT NULL UNIQUE
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE incidents (
incident_id SERIAL PRIMARY KEY,
title VARCHAR(255) NOT NULL,
description TEXT NOT NULL,
status_id INT NOT NULL,
severity_id INT NOT NULL,
reporter_id INT NOT NULL,
assigned_engineer_id INT,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_incident_status
FOREIGN KEY (status_id)
REFERENCES incident_statuses(status_id),
CONSTRAINT fk_incident_severity
FOREIGN KEY (severity_id)
REFERENCES incident_severities(severity_id),
CONSTRAINT fk_incident_reporter
FOREIGN KEY (reporter_id)
REFERENCES users(user_id),
CONSTRAINT fk_incident_engineer
FOREIGN KEY (assigned_engineer_id)
REFERENCES users(user_id)
ON DELETE SET NULL
);
```

Data Output | Messages | Notifications

```
CREATE TABLE
```

Query | Query History

```sql
CREATE TABLE incident_services (
incident_id INT NOT NULL,
service_id INT NOT NULL,
is_primary BOOLEAN NOT NULL DEFAULT FALSE,
PRIMARY KEY (incident_id, service_id),
CONSTRAINT fk_incident_services_incident
FOREIGN KEY (incident_id)
REFERENCES incidents(incident_id)
ON DELETE CASCADE,
CONSTRAINT fk_incident_services_service
FOREIGN KEY (service_id)
REFERENCES services(service_id)
ON DELETE CASCADE
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE incident_events (
event_id SERIAL PRIMARY KEY,
incident_id INT NOT NULL,
user_id INT,
event_type VARCHAR(100) NOT NULL,
event_description TEXT NOT NULL,
event_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_event_incident
FOREIGN KEY (incident_id)
REFERENCES incidents(incident_id)
ON DELETE CASCADE,
CONSTRAINT fk_event_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE SET NULL
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE deployments (
deployment_id SERIAL PRIMARY KEY,
service_id INT NOT NULL,
version VARCHAR(100) NOT NULL,
environment VARCHAR(50) NOT NULL,
deployed_by INT,
deployed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
deployment_status VARCHAR(30) NOT NULL,
CONSTRAINT fk_deployment_service
FOREIGN KEY (service_id)
REFERENCES services(service_id)
ON DELETE CASCADE,
CONSTRAINT fk_deployment_user
FOREIGN KEY (deployed_by)
REFERENCES users(user_id)
ON DELETE SET NULL,
CONSTRAINT chk_deployment_status
CHECK (deployment_status IN ('SUCCESS', 'FAILED', 'ROLLED_BACK'))
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE incident_deployments (
incident_id INT NOT NULL,
deployment_id INT NOT NULL,
PRIMARY KEY (incident_id, deployment_id),
CONSTRAINT fk_incident_deployment_incident
FOREIGN KEY (incident_id)
REFERENCES incidents(incident_id)
ON DELETE CASCADE,
CONSTRAINT fk_incident_deployment_deployment
FOREIGN KEY (deployment_id)
REFERENCES deployments(deployment_id)
ON DELETE CASCADE
);

```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE resolutions (
resolution_id SERIAL PRIMARY KEY,
incident_id INT NOT NULL UNIQUE,
root_cause_category VARCHAR(100) NOT NULL,
root_cause_description TEXT NOT NULL,
resolution_summary TEXT NOT NULL,
resolved_by INT,
resolved_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_resolution_incident
FOREIGN KEY (incident_id)
REFERENCES incidents(incident_id)
ON DELETE CASCADE,
CONSTRAINT fk_resolution_user
FOREIGN KEY (resolved_by)
REFERENCES users(user_id)
ON DELETE SET NULL
);

```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE TABLE audit_logs (
audit_id SERIAL PRIMARY KEY,
user_id INT,
action VARCHAR(100) NOT NULL,
entity_name VARCHAR(100) NOT NULL,
entity_id INT NOT NULL,
old_values JSONB,
new_values JSONB,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_audit_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE SET NULL
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 234 msec.
```

Query | Query History

```sql
CREATE INDEX idx_incidents_status
ON incidents(status_id);

CREATE INDEX idx_incidents_severity
ON incidents(severity_id);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 97 msec.
```

Query | Query History

```sql
CREATE INDEX idx_incidents_reporter
ON incidents(reporter_id);

CREATE INDEX idx_incidents_assigned_engineer
ON incidents(assigned_engineer_id);

CREATE INDEX idx_incidents_created_at
ON incidents(created_at);

CREATE INDEX idx_incident_events_incident_time
ON incident_events(incident_id, event_timestamp);

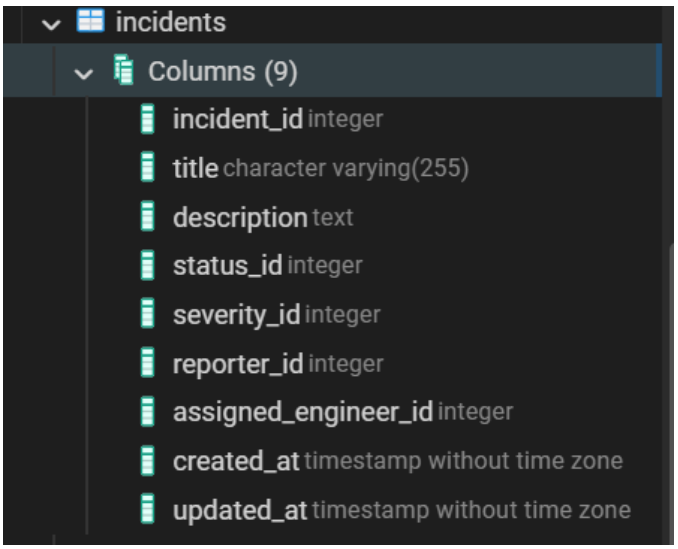
CREATE INDEX idx_deployments_service_date
ON deployments(service_id, deployed_at);

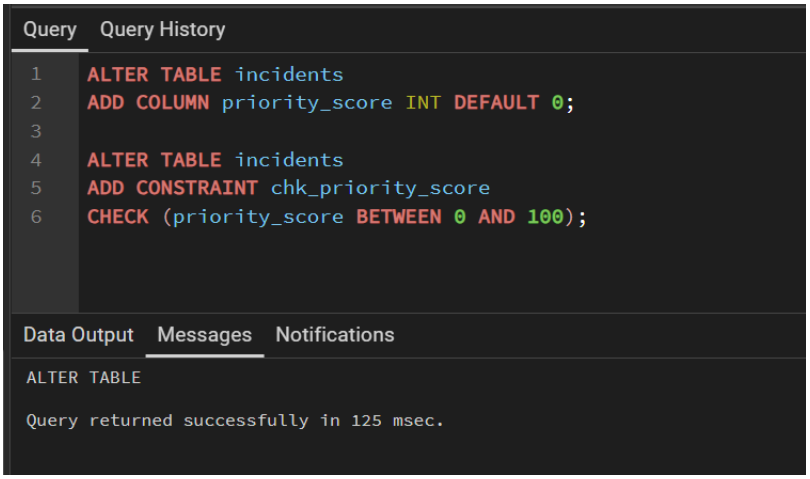
CREATE INDEX idx_service_dependencies_service
ON service_dependencies(service_id);
```

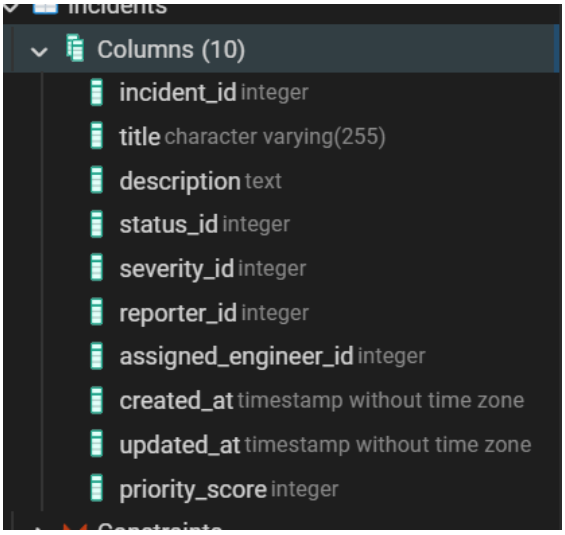
Data Output | Messages | Notifications

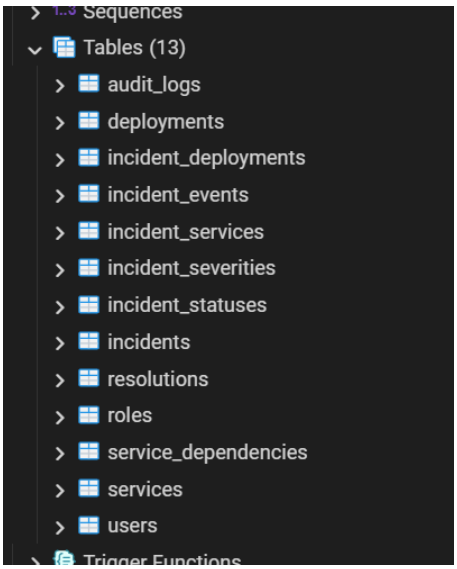
```
CREATE INDEX

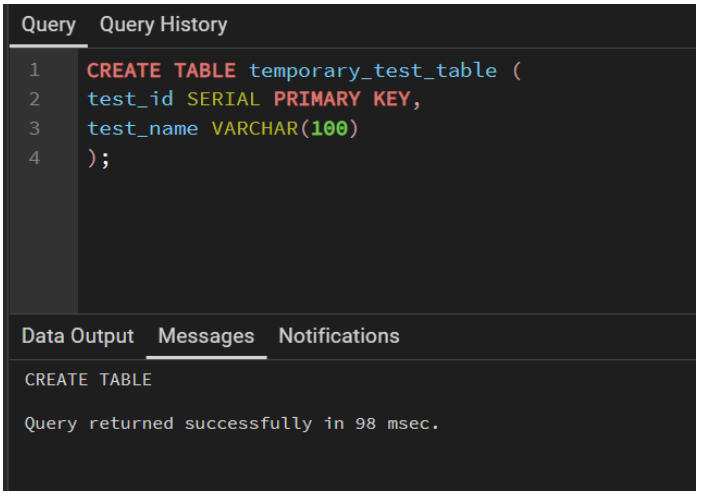
Query returned successfully in 80 msec.
```

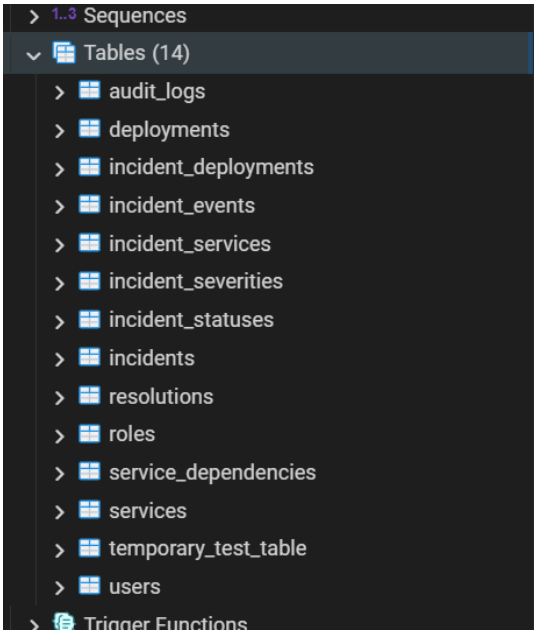

incidents
Columns (9)
incident_id integer
title character varying(255)
description text
status_id integer
severity_id integer
reporter_id integer
assigned_engineer_id integer
created_at timestamp without time zone
updated_at timestamp without time zone


Query
Query History
1 ALTER TABLE incidents
2 ADD COLUMN priority_score INT DEFAULT 0;
3
4 ALTER TABLE incidents
5 ADD CONSTRAINT chk_priority_score
6 CHECK (priority_score BETWEEN 0 AND 100);
Data Output
Messages
Notifications
ALTER TABLE
Query returned successfully in 125 msec.


Columns (10)
incident_id integer
title character varying(255)
description text
status_id integer
severity_id integer
reporter_id integer
assigned_engineer_id integer
created_at timestamp without time zone
updated_at timestamp without time zone
priority_score integer


Sequences
Tables (13)
audit_logs
deployments
incident_deployments
incident_events
incident_services
incident_severities
incident_statuses
incidents
resolutions
roles
service_dependencies
services
users
Trigger Functions


Query
Query History
1 CREATE TABLE temporary_test_table (
2 test_id SERIAL PRIMARY KEY,
3 test_name VARCHAR(100)
4);
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 98 msec.


Sequences
Tables (14)
audit_logs
deployments
incident_deployments
incident_events
incident_services
incident_severities
incident_statuses
incidents
resolutions
roles
service_dependencies
services
temporary_test_table
users
Trigger Functions

Query Query History

```
1  DROP TABLE temporary_test_table;
```

Data Output Messages Notifications

```
DROP TABLE

Query returned successfully in 77 msec.
```

